56

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Spring Boot CI Pipeline: Maven Test, Build ve Kalite Kontrolü

Spring Boot CI Pipeline: Maven Test, Build & Quality Gates

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Hatalı bir kodun üretim hattına geçmesini engelleyen acımasız kapı bekçisi! Yazılımcı gece yarısı dalgınlıkla bir hataya sebep olduysa veya birim testleri (Unit Tests) bozduysa, GitHub Actions o kodu anında durdurur: Kırmızı çarpı işaretini basar, yöneticilere haber verir ve bozuk kodun Docker imajı olmasına ASLA izin vermez!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The unyielding quality gatekeeper! If a sleepy engineer accidentally introduces a regression or breaks a unit test, GitHub Actions slams the brakes on the assembly line: it raises a red flag, alerts the engineering team, and strictly prevents the broken build from ever producing a container image!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Etkili bir Java CI adımı şunları içerir: 1) `actions/setup-java@v4`: Eclipse Temurin JDK 21 kurulumu ve `cache: 'maven'` ile `~/.m2` deposunun önbelleğe alınması (derleme süresini 4 dakikadan 30 saniyeye düşürür). 2) `mvn clean test`: Tüm JUnit 5 ve Spring Boot MockMvc testlerini çalıştırır. 3) Test raporlarının ve kod kapsama oranlarının (JaCoCo) üretilmesi. 4) `mvn package -DskipTests`: Üretim JAR dosyasının derlenmesi.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A resilient Spring Boot CI pipeline includes: 1) `setup-java` configuring JDK 21 with automatic Maven dependency caching (shrinking build times from minutes to seconds). 2) `mvn clean test` executing JUnit 5 unit and integration test suites. 3) JaCoCo test coverage enforcement. 4) Packaging the final layered production executable JAR via Spring Boot Maven plugin.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# Maven CI Job Bloğu:
jobs:
  test-and-package:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: JDK 21 Temurin Kur ve Maven Önbellekle
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: maven

      - name: Birim ve Entegrasyon Testlerini Çalıştır
        run: mvn clean test

      - name: Üretim JAR Paketini Derle
        run: mvn package -DskipTests
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Maven önbellekleme (`cache: maven`) yapmazsanız, GitHub Actions her commit'te tüm Spring Boot kütüphanelerini internetten sıfırdan indirir; bu da dakikalarca zaman kaybına ve GitHub Actions dakikalarınızın hızla tükenmesine yol açar.

🇬🇧 WATCH OUT

Omitting Maven dependency caching forces the runner to re-download hundreds of megabytes of dependencies on every push, wasting pipeline minutes and developer time.

🎯 Özet Çıkarımlar

- Otomatik testler bozuk kodların müşterilere ulaşmasını %100 engeller.
- Maven önbellekleme derleme sürelerini kat kat hızlandırır.
- Test raporları her Pull Request üzerinde şeffaf biçimde incelenebilir.

🎯 Key Takeaways

- Automated CI testing guarantees that failing code cannot slip unnoticed into production.
- Maven dependency caching slashes build times from minutes down to seconds.
- Test outcomes are surfaced directly on GitHub Pull Requests for peer review.